ЛАБОРАТОРНАЯ РАБОТА 3

Распределённый анализ вакансий DevOps

RustFS, MapReduce и аналитика DuckDB

Вариант 11 | Смирнов Александр | J3310
18 октября 2026 года

Цель работы

Подготовить сегмент вакансий DevOps, хранить данные в объектном S3-совместимом хранилище, сравнить собственный MapReduce с SQL в DuckDB и исследовать навыки, зарплаты и региональные особенности.

Результат

Из 229 388 вакансий выбрано 6 215. Для них рассчитаны частоты 1 541 навыка, 25 462 пар и 195 366 троек. Вычисления MapReduce совпали с независимым запросом DuckDB по каждому ключу и каждой частоте.

Основной этап	Полученный результат
Подготовка данных	6 215 записей NDJSON; контрольный SHA-256
Хранилище	RustFS, бакет hh-lake; результаты доступны по S3 API
Подсчёт	MapReduce с combiner и независимый DuckDB httpfs
Проверка	Полное совпадение трёх JSON; 3 запуска каждого метода
Аналитика	Зарплатные сочетания и три региона с наибольшим числом вакансий

Данные и формирование сегмента

Исходный файл - CSV с вакансиями IT-специалистов. Отбор выполнялся по названию позиции. В сегмент включены DevOps и DevSecOps, русское написание «девопс», SRE, Site Reliability и Platform Engineer. Один SQL-запрос к исходному CSV проверяет тот же критерий независимо от Python.

Показатель	Значение
Исходный файл	it_vacancies_2024.csv
Размер	847 619 384 байт
SHA-256	303db922b541e4440906f5a7e8be32fb55e8808f950ffddb8c272f1548212a65
Записей в исходном CSV	229 388
Выбрано для анализа	6 215
Из выбранных: с навыками	4 266 (68.6%)
Сопоставимая зарплата в RUR	1 364 (21.9%)
Дата публикации	2023-12-19 - 2024-09-14

Независимый SQL отобрал 6 215 строк; SQL-only = 0, prepared-only = 0. SHA-256 обоих отсортированных наборов ID совпал: 054c2ede9b76850e3b995d2df9fdbbf2353baa9bd5ba60b08bd6268a0dae768e.

Подготовка

Каждая строка NDJSON соответствует одной вакансии. Навыки приведены к нижнему регистру, пробелы нормализованы, повторы удалены. Сохраняются идентификатор, дата, регион, исходные границы зарплаты и оценка зарплаты gross в рублях. Файл сегмента имеет размер 1 880 228 байт; SHA-256: 5c9daf1ac2c16c3aad36d97e6e23c7cffc6d92f0cf4ca215d73f8140e721b0c7.



Рисунок 1. Наиболее частые навыки в отобранных вакансиях.

Объектное хранилище

RustFS запущен как локальный S3-совместимый сервис. В бакете hh-lake размещены исходный CSV, подготовленный NDJSON, журналы и результаты. Для всех трёх выходных JSON выполнена повторная загрузка из бакета; проверены количество байт и SHA-256 фактически прочитанного содержимого.

Ключ результата	Байт	SHA-256 скачанного объекта
skills.json	141 664	c18b9a9352795ce42d3aa7d5ef845af43083daa0f020e51e43e4ccda6730e7d0
pairs.json	2 881 790	ba6dcc751253937d2322bfc7d2985c387f113149c7bed7e73b7030fea69e512e
triples.json	26 156 464	0fb6cddf70e5c20ecc0f6ea8220de0cd31867dfb9a0eea1873f41afc173b0edb

S3 ETag не используется как SHA-256: для multipart-объектов он имеет другой формат. Вместо этого сравниваются метаданные загрузки и хеш потока, скачанного обратно из S3.

Интерфейс RustFS

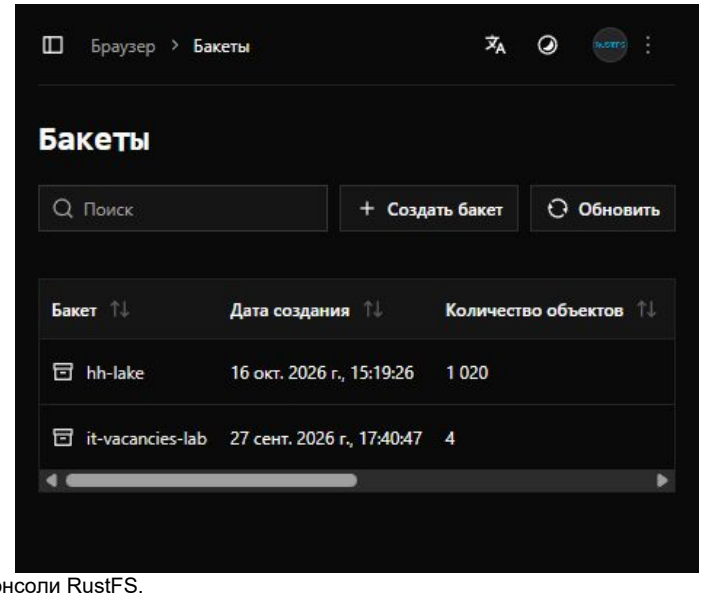


Рисунок 2. Бакет hh-lake в консоли RustFS.

Содержимое бакета

Пространство имён разделяет исходные и производные данные. Такой порядок помогает повторно использовать исходный файл, не смешивая его с результатами вычислений.

Префикс	Содержимое
raw/	Исходный CSV
variants/devops/	NDJSON-сегмент и метаданные подготовки
results/mapreduce/	Три результата навыков, пар и троек
results/duckdb/	Независимые SQL-результаты, зарплаты и регионы
evidence/	Инвентаризация и журналы проверок
supplementary/	Выборка JSON-вакансий

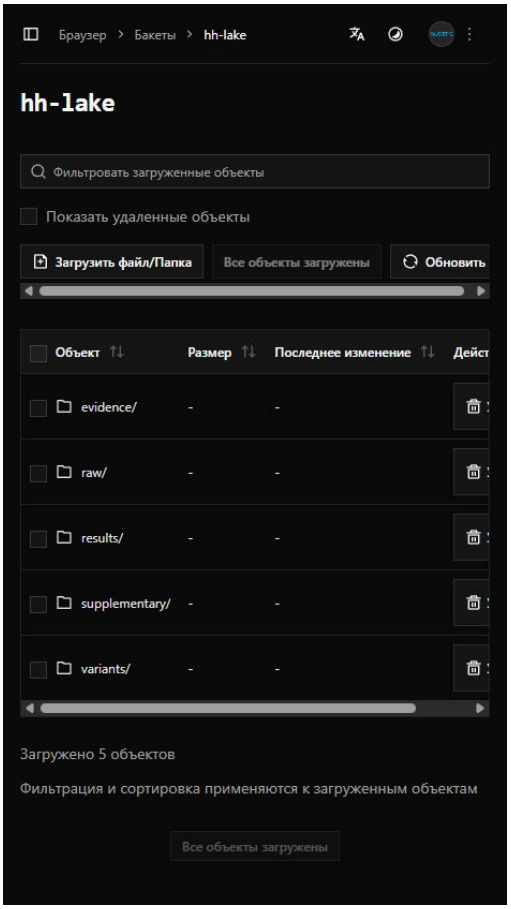


Рисунок 3. Префиксы бакета hh-lake в консоли RustFS.

Обработка чанков и подсчёт

Размер S3-объекта получают через HEAD. Файл делится на непрерывные байтовые диапазоны. Mapper читает каждый диапазон запросом Range GET.

Ситуация на границе	Действие Mapper
Диапазон начинается внутри строки	Пропустить байты до следующего перевода строки
Диапазон заканчивается внутри строки	Дочитать запись блоками по 64 КиБ
Строка длиннее 64 КиБ	Продолжать дочитывание до конца записи
Последняя строка без \n	Обработать оставшиеся байты объекта
Ошибка JSON или структура навыков нарушена	Завершить с номером чанка и причиной

Чанк, содержащий первый байт записи, отвечает за всю эту запись. Поэтому дочитанные соседние байты не приводят к двойному учёту. В памяти комбинации представлены кортежами навыков; отдельный символ-разделитель не нужен. В итоговом JSON пары и тройки сериализуются как списки.

В baseline Mapper передаёт все комбинации. В режиме с combiner он складывает одинаковые комбинации в локальный Counter и передаёт счётчики Reducer. Reducer объединяет оба формата одинаковым вызовом update().

Восстановление

В тестовом прогоне намеренно отказал второй Mapper. Координатор повторил один чанк: retries = 1. После повтора обработаны 6 215 вакансий; все три результата совпали с обычным запуском. Повторяются временные сетевые ошибки и коды S3/HTTP, обозначающие временный сбой. Ошибки данных повторно не выполняются.

Независимый подсчёт и проверка

DuckDB читает тот же NDJSON из RustFS через расширение httpfs. Массив навыков разворачивается в строки; для пар и троек используются самосоединения по vacancy_id и лексикографическое условие skill1 < skill2 < skill3. Таким образом, Python MapReduce и SQL получают комбинации независимо.

Стек	Уникальные ключи	Сумма частот	Результат
Навыки	1 541	35 158	совпало
Пары	25 462	173 204	совпало
Тройки	195 366	687 102	совпало

Сопоставлены полные JSON-документы: метаданные, порядок ключей и частота каждой комбинации. SHA-256 канонических результатов:

Результат	SHA-256
skills	2ad809f9e9da9c73c8d311ace571b27b367b28cdf96b56d7e892e19615957e58
pairs	cafaec2c23af084d67ce40c8cf4e7847ab46bf54766cb1663d1213d500877a12
triples	4f194e88fc2c73d40541285a53d131d81b10b73f29fb257f9eaa01a9cd8bb4c2

S3 API перечислил 3 объекта в префиксе results/mapreduce/. Там находятся skills.json, pairs.json и triples.json. Результаты не ограничены первой строкой или top-N: полные частотные таблицы сохранены в хранилище.

Результаты MapReduce в консоли RustFS

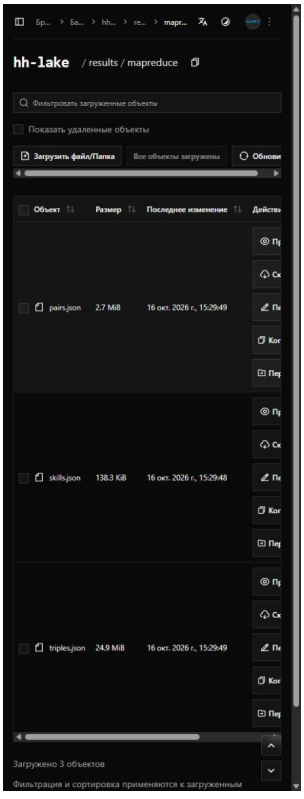


Рисунок 4. Объекты skills.json, pairs.json и triples.json.

Частота навыков и сочетаний

Число уникальных комбинаций растёт с размером стека, потому что каждая вакансия создаёт сочетания навыков. Для k уникальных навыков количество одиночных навыков равно k, пар - C(k,2), троек - C(k,3). При десяти навыках вакансия порождает 10 навыков, 45 пар и 120 троек.

Размер стека	Уникальные	Всего в вакансиях	На одну вакансию
1 навык	1 541	35 158	k
2 навыка	25 462	173 204	k(k - 1) / 2
3 навыка	195 366	687 102	k(k - 1)(k - 2) / 6

«Уникальные» считает разные ключи во всём сегменте. «Всего» суммирует вхождения ключа по вакансиям. Одна и та же пара, найденная в тысяче вакансий, вносит одну единицу в число уникальных пар и тысячу в общую частоту.

Наиболее частые навыки



Рисунок 5. Десять самых частых навыков; частота указана в вакансиях.

Сравнение времени и памяти

Каждый метод запускался трижды на одном объекте RustFS, с четырьмя процессами MapReduce или четырьмя потоками DuckDB и чанком 524 288 байт. Время включает запуск процесса, S3-чтение и запись результатов. В таблице приведены отдельные попытки.

Метод	Запуск	Wall time, с	Пик RSS, МиБ
MapReduce baseline	1	2.221	410.7
MapReduce baseline	2	2.030	411.2
MapReduce baseline	3	2.098	420.4
MapReduce + combiner	1	2.047	373.1
MapReduce + combiner	2	2.074	363.7
MapReduce + combiner	3	1.988	366.3
DuckDB через httpfs	1	1.047	331.0
DuckDB через httpfs	2	1.046	323.9
DuckDB через httpfs	3	1.061	322.8

Метод	Медиана времени, с	Медиана RSS, МиБ
MapReduce baseline	2.098	411.2
MapReduce + combiner	2.047	366.3
DuckDB через httpfs	1.047	323.9

Combiner сократил число передаваемых элементов с 895 464 до 342 787. Размер сериализованных payload уменьшился с 14 149 696 до 6 316 587 байт (55.4%). Медиана времени MapReduce изменилась с 2.098 до 2.047 с; пик RSS - с 411.2 до 366.3 МиБ.

В этом запуске DuckDB завершил расчёт за 1.047 с при пике 323.9 МиБ. Такой результат объясним эффективным векторизованным выполнением SQL на небольшом наборе. Профилирование характеризует этот стенд: RSS снимался каждые 10 мс, а общие страницы процессов могли учитываться повторно.

Запуски выполнялись последовательно без очистки системного кеша. Числа полезны для сравнения двух реализаций в указанных условиях; они не предсказывают скорость другого компьютера или удалённого кластера.

Навыки и медианная зарплата

Для 1 364 вакансий с известной месячной зарплатой в RUR общая медиана gross равна 229 885 руб. При двух границах зарплатного диапазона используется середина, при одной - указанная граница. Значения net переводятся в gross делением на 0,87. Иностранные валюты исключаются.

Для пары базовым считается более востребованный одиночный навык. Для тройки выбирается наиболее востребованная входящая пара. Разница медиан равна медиане комбинации минус медиана базы. В публикацию включены пары минимум с 20 зарплатными наблюдениями и тройки минимум с 15.

Пары с наибольшей разницей

Сочетание	База	n	Медиана базы	Медиана сочетания	Разница, руб.
devops + terraform	devops	62	229 885	344 828	+114 943
terraform + zabbix	zabbix	35	229 885	344 828	+114 943
devops + elk	devops	75	229 885	333 333	+103 448
debian + nginx	nginx	24	219 195	114 943	-104 253
linux + администрирование серверов	linux	20	229 885	126 437	-103 448
debian + linux	linux	32	229 885	128 966	-100 920

Тройки с наибольшей разницей

Сочетание	База	n	Медиана базы	Медиана сочетания	Разница, руб.
bash + ci/cd + elk	bash + ci/cd	19	235 632	402 299	+166 667
bash + devops + elk	bash + devops	27	244 253	402 299	+158 046
ci/cd + elk + gitlab	ci/cd + gitlab	20	252 874	388 649	+135 776
bash + kubernetes + mysql	bash + kubernetes	21	287 356	172 414	-114 943
bash + nginx + sql	bash + nginx	19	224 138	114 943	-109 195
linux + sql + zabbix	linux + zabbix	15	221 264	114 943	-106 322

Положительная разница медиан показывает связь между сочетанием и более высокой наблюдаемой зарплатой. Она не доказывает, что добавление навыка повышает оплату: роли, стаж, отрасль и работодатель влияют на зарплату, но здесь отдельно не контролируются. Наиболее отрицательные сочетания также приведены, чтобы показать обе стороны распределения.

Навыки записаны работодателями в свободной форме, раскрытие оплаты неполное. Поэтому медианы описывают вакансии с опубликованными суммами, а не всех специалистов или весь рынок.

Москва

В выборке 3 536 вакансий; навыки указаны в 2 355. Известная зарплата есть у 699. Медиана gross - 287 356 руб.; в среднем 5.25 навыка на вакансию.

В каждой строке ниже частота - число вакансий, где указана вся комбинация. Доля с навыками рассчитана от вакансий с непустым списком навыков; доля от всех вакансий использует полный размер сегмента региона.

Пять наиболее частых пар

Сочетание	Вакансий	Доля с навыками, %	Доля всех, %
docker + kubernetes	860	36.52	24.32
docker + linux	816	34.65	23.08
kubernetes + linux	810	34.39	22.91
devops + linux	571	24.25	16.15
ansible + linux	561	23.82	15.87

Пять наиболее частых троек

Сочетание	Вакансий	Доля с навыками, %	Доля всех, %
docker + kubernetes + linux	558	23.69	15.78
devops + docker + kubernetes	382	16.22	10.80
devops + kubernetes + linux	373	15.84	10.55
ansible + docker + linux	366	15.54	10.35
ansible + kubernetes + linux	355	15.07	10.04

Что видно в выборке

Пара Docker + Kubernetes указана в 860 вакансиях; тройка с Linux - в 558. У Москвы больше наблюдений и зарплатных раскрытий, чем у остальных выбранных регионов. Высокая медиана может быть связана с составом работодателей и ролей; причинные факторы этим сравнением не проверяются.

Популярность описывает опубликованные требования. Она не показывает глубину навыка, качество рабочего места или причинный эффект на зарплату.

Санкт-Петербург

В выборке 841 вакансий; навыки указаны в 593. Известная зарплата есть у 217. Медиана gross - 229 885 руб.; в среднем 5.96 навыка на вакансию.

В каждой строке ниже частота - число вакансий, где указана вся комбинация. Доля с навыками рассчитана от вакансий с непустым списком навыков; доля от всех вакансий использует полный размер сегмента региона.

Пять наиболее частых пар

Сочетание	Вакансий	Доля с навыками, %	Доля всех, %
docker + linux	223	37.61	26.52
docker + kubernetes	199	33.56	23.66
ansible + docker	175	29.51	20.81
ansible + linux	174	29.34	20.69
kubernetes + linux	173	29.17	20.57

Пять наиболее частых троек

Сочетание	Вакансий	Доля с навыками, %	Доля всех, %
docker + kubernetes + linux	128	21.59	15.22
ansible + docker + linux	119	20.07	14.15
ansible + docker + kubernetes	100	16.86	11.89
ansible + kubernetes + linux	95	16.02	11.30
ci/cd + docker + linux	83	14.00	9.87

Что видно в выборке

Самая частая пара - Docker + Linux (223 вакансии); далее Docker + Kubernetes (199). Ansible встречается в четырёх из пяти самых частых троек. Доля раскрытых зарплат выше московской, поэтому медианы основаны на разных частях выборки.

Популярность описывает опубликованные требования. Она не показывает глубину навыка, качество рабочего места или причинный эффект на зарплату.

Екатеринбург

В выборке 231 вакансий; навыки указаны в 164. Известная зарплата есть у 43. Медиана gross - 183 908 руб.; в среднем 6.14 навыка на вакансию.

В каждой строке ниже частота - число вакансий, где указана вся комбинация. Доля с навыками рассчитана от вакансий с непустым списком навыков; доля от всех вакансий использует полный размер сегмента региона.

Пять наиболее частых пар

Сочетание	Вакансий	Доля с навыками, %	Доля всех, %
docker + kubernetes	71	43.29	30.74
kubernetes + linux	71	43.29	30.74
docker + linux	66	40.24	28.57
ansible + linux	49	29.88	21.21
ansible + kubernetes	48	29.27	20.78

Пять наиболее частых троек

Сочетание	Вакансий	Доля с навыками, %	Доля всех, %
docker + kubernetes + linux	53	32.32	22.94
ansible + kubernetes + linux	41	25.00	17.75
ansible + docker + kubernetes	39	23.78	16.88
ansible + docker + linux	37	22.56	16.02
devops + docker + kubernetes	37	22.56	16.02

Что видно в выборке

Две ведущие пары встречаются одинаково часто: Docker + Kubernetes и Kubernetes + Linux, по 71 вакансии каждая. Основная тройка Docker + Kubernetes + Linux есть в 53 вакансиях с навыками. Высокая доля сочетается с небольшой абсолютной выборкой и 43 зарплатными наблюдениями; эту медиану следует интерпретировать осторожно.

Популярность описывает опубликованные требования. Она не показывает глубину навыка, качество рабочего места или причинный эффект на зарплату.

Выводы и ограничения

MapReduce по S3 Range GET обработал данные по чанкам; combiner сократил передаваемые повторяющиеся комбинации. Независимые SQL-запросы DuckDB к тому же NDJSON дали те же частоты навыков, пар и троек. S3 API подтвердил наличие результатов в hh-lake и целостность скачанного содержимого.

На этом объёме DuckDB был быстрее и использовал меньший пик RSS. Для лабораторной ценность MapReduce состоит в демонстрации распределения задач и combiner; производительность зависит от размера данных, числа чанков, запросов к S3 и сериализации.

В трёх регионах повторяются Linux, Docker и Kubernetes. Москва имеет больше всего вакансий и зарплатных наблюдений; в Екатеринбурге абсолютные частоты меньше, хотя доля ведущей тройки среди вакансий с навыками выше. Неполное раскрытие зарплат и малый региональный размер не позволяют трактовать медианы как гарантированные рыночные уровни.

Ограничения

Фактор	Как учитывать при интерпретации
Оплата раскрыта не для всех вакансий	Зарплатная выборка не является случайной
Свободный текст названий и навыков	Фильтр может пропускать и включать пограничные случаи
Диапазон и net-to-gross 13%	Оценка сопоставимости, а не расчёт зарплаты сотрудника
Медианная разница	Ассоциация; стаж, роль и работодатель не контролируются
Региональный состав	Частоты и медианы зависят от размера и структуры выборки
Память процесса	Опрос раз в 10 мс; общие страницы могут учитываться повторно

Воспроизведение

Подготовить CSV и RustFS согласно README. Выполнить scripts/validate_rustfs.ps1 для загрузки входных данных, трёх расчётов, проверки S3, восстановления Mapper и профилирования. JSON-результаты и параметры запуска сохраняются в evidence/rustfs-validation/.

Источники

Материал	Описание
Исходный CSV	it_vacancies_2024.csv; SHA-256 303db922b541e4440906f5a7e8be32fb55e8808f950ffddb8c272f1548212a65
Подготовленный NDJSON	devops_vacancies.ndjson; SHA-256 5c9daf1ac2c16c3aad36d97e6e23c7cffc6d92f0cf4ca215d73f8140e721b0c7
Документация RustFS	https://docs.rustfs.com/
DuckDB httpfs: S3 API	https://duckdb.org/docs/stable/core_extensions/httpfs/s3api
DuckDB: чтение JSON	https://duckdb.org/docs/stable/data/json/loading_json
Python: itertools.combinations	https://docs.python.org/3/library/itertools.html#itertools.combinations
S3 API	https://docs.aws.amazon.com/AmazonS3/latest/API/API_GetObject.html

Расчёты выполнены для варианта 11 с фильтром DevOps, DevSecOps, девопс, SRE, Site Reliability и Platform Engineer. Формулировка отбора приведена в разделе данных.